



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Faculty of Computer Science & Engineering

MicroLend

Community Microfinance Loan Servicing and Delinquency Engine

SE431 — Software Quality Assurance

Milestone 1 — Quality Requirements, RTM and SQA Plan

Submitted by

Muhammad Ibrahim	2023446
Hassan Khalid	2023242
Tughral Hussain	2023532

Instructor: Dr. Mian Muaz Razaq

Fall 2026

Contents

MicroLend — Community Microfinance Loan Servicing & Delinquency Engine	1
1. Quality Requirements (ISO/IEC 25010)	1
2. Requirements Traceability Matrix — Version 1	3
3. SQA Plan — Scope and Outline (IEEE 730)	6
4. Cost of Quality and Quality-Culture Statement	8
5. Standards Awareness	10
6. Initial Risk Register	10
7. Summary of M1 Quality Position	14
Annex A — AI Usage Log	15
Declaration for Milestone 1	15
Team rules for AI use	15

MicroLend — Community Microfinance Loan Servicing & Delinquency Engine

Team

Member	Reg. #	M1 SQA deliverable lead	Construction modules
Muhammad Ibrahim	2023446	SQAP scope & outline, risk register	origination, delinquency
Hassan Khalid	2023242	Quality requirements (ISO/IEC 25010), RTM v1	products, reporting
Tughrul Hussain	2023532	Standards awareness, Cost of Quality	ledger

Repository: <https://github.com/tughrul1/microlend> **Companion document:** docs/M1-SE423-Proposal-00D-Pattern-Plan.md — the construction-side proposal this quality plan applies to **Milestone:** M1 (Week 6) · **Weight:** 15% of the project grade

1. Quality Requirements (ISO/IEC 25010)

1.1 Which characteristics matter for this system, and which do not

ISO/IEC 25010 defines eight product quality characteristics. A quality plan that claims all eight matter equally has not analysed the system. MicroLend’s product risk is **financial, not safety-related**, and it is an internal back-office tool with a handful of users, which determines where quality effort is spent:

Characteristic	Relevance to MicroLend	Why
Functional suitability	Critical	A schedule or allocation that is arithmetically wrong corrupts every downstream figure — balances, penalties, classification, provisioning, reports
Reliability	Critical	A failure part-way through posting must not leave a half-written ledger; the end-of-day engine must survive a task dying
Security (integrity)	High	A ledger that can be altered is not an audit record. Integrity, not confidentiality, is the sub-characteristic that matters
Maintainability	High	The system is assessed across three milestones by three developers who must each work in code the others wrote
Performance efficiency	Moderate	The end-of-day run must finish within an overnight window. Nothing else is time-critical
Usability	Low	A small internal web interface used by trained branch staff on a local machine. Learnability is not a project risk

Characteristic	Relevance to MicroLend	Why
Compatibility	Not applicable	A single self-contained application; no co-existence or interoperability requirement
Portability	Not applicable	Runs locally on the JVM with a file-based database and is never deployed, so there is no deployment-environment variation in scope

Compatibility and portability are declared **not applicable with justification** rather than quietly omitted.

1.2 The quality requirements

Each requirement carries exactly one measurable claim and names one of the four verification methods — **inspection**, **analysis**, **demonstration**, **test**.

ID	Requirement	Characteristic	Sub-characteristic	Measurable target	Verification method and evidence
QR-01	End-of-day processing of 10,000 active loans shall complete within 20 seconds on the demo machine using 4 threads	Performance efficiency	Time behaviour	Median wall-clock over 5 consecutive runs ≤ 20 s; no single run > 25 s	Test — timed JUnit harness over a generated 10,000-loan dataset; timings attached to each milestone report
QR-02	Repayment allocation shall reproduce the reference schedules exactly, at the stated monetary scale	Functional suitability	Functional correctness	Zero discrepancy against 30 hand-computed golden schedules at scale 2, HALF_EVEN; 100% decision coverage of <code>RepaymentA1-locator.post()</code>	Test — golden-file unit tests plus a JaCoCo branch-coverage report
QR-03	Any failure during repayment posting shall leave the ledger balanced, with no partial entry	Reliability	Fault tolerance	For 100% of at least 10 injected failure points: $\Sigma \text{ debits} = \Sigma \text{ credits}$, and zero orphan entries	Test — fault injection through a repository stub that throws at each write point; also demonstrated live at M3 as one of the two required failure scenarios
QR-04	Core business-logic methods shall remain within the stated complexity and size limits	Maintainability	Analysability / Modifiability	Mean cyclomatic complexity ≤ 5 ; no method above 15; no class above 200 LOC, measured across the five module packages	Analysis — static-analysis metric report produced in CI and attached to each milestone

ID	Requirement	Characteristic	Sub-characteristic	Measurable target	Verification method and evidence
QR-05	A posted ledger entry shall not be alterable by any component; corrections shall occur only as compensating entries	Security	Integrity	Zero mutating members on <code>LedgerEntry</code> ; zero <code>UPDATE</code> statements against <code>ledger_entry</code> in the codebase; 100% of corrections implemented as compensating entries	Inspection — an explicit item on the Fagan inspection checklist, supported by a static-analysis rule and a unit test asserting reversal-only correction
QR-06	The end-of-day engine shall produce identical results whether or not repayments are posted concurrently during the run	Reliability	Maturity	Over 50 repeated runs of the concurrency suite: zero lost updates, zero double accruals, identical final balances to the single-threaded reference run	Test — deterministic latch-driven concurrency tests; the suite is repeated 50 times nightly in CI

1.3 Constraints (not quality requirements)

The standard separates requirements into functional, non-functional (quality) and **constraints**. The following are constraints on how the system is built. They are recorded in SQAP §4.4 rather than presented as quality requirements:

ID	Constraint	Rationale
C-01	Monetary values shall never be represented as double or float	$0.1 + 0.2 \neq 0.3$ in IEEE 754; a fraction of a rupee per instalment accumulates into a ledger that does not balance. Enforced by a Checkstyle rule that fails the build
C-02	<code>BigDecimal</code> values shall be compared with <code>compareTo() == 0</code> , never <code>equals()</code>	<code>equals()</code> also compares scale, so <code>2.50</code> is not equal to <code>2.5</code>
C-03	No cyclic dependencies between module packages	Cycles destroy the ability to test or replace a module in isolation
C-04	Monetary columns shall be stored as integer minor units	Same reasoning as C-01, applied at the storage layer

1.4 The quality trade-off

QR-01 (performance) is in tension with QR-03, QR-05 and QR-06 (fault tolerance, integrity, maturity). Per-account locking, immutable value objects, append-only correction and an idempotency key all cost time behaviour: each adds work on the path that QR-01 measures.

The conflict is resolved by **capping the cost rather than removing either attribute** — locks are per account rather than global, so unrelated accounts proceed in parallel and the lock is never held across I/O. The measured cost is recorded in the GQM data at each milestone rather than asserted to be small.

2. Requirements Traceability Matrix — Version 1

2.1 Purpose and how it will be maintained

The RTM supports traceability in **both directions**. Forward — from requirement to code to test — answers “*did we build and verify everything that was asked for?*” and exposes a **test gap**: a requirement implemented but never verified. Backward — from code to requirement — answers “*why does this code exist?*” and exposes **rogue code**: a module traceable to no approved requirement.

Those are the two defects this matrix is audited for at every milestone. At M1 the design and class columns are planned rather than built, so they are marked *planned*; from M2 they name real artifacts and real test results.

The matrix is maintained at `qa/rtm.md` in the repository and versioned with the code, so the state of traceability at any baseline can be reproduced from the tag.

2.2 Functional requirements

ID	Requirement	Module	Design element (planned)	Test approach	Test design technique
FR-01	An applicant shall be at least 18 years old at disbursement and no more than 65 at maturity	origination	AgeRule	Unit	BVA: 17 / 18 / 65 / 66 at both dates
FR-02	An applicant shall hold no more than 2 active loans	origination	ActiveLoanLimitRule	Unit	EP + BVA: 0, 1, 2, 3 active loans
FR-03	Total monthly instalment burden shall not exceed 40% of declared monthly income	origination	BurdenRatioRule	Unit	BVA: 39.99% / 40.00% / 40.01%
FR-04	An applicant with a prior write-off shall be rejected	origination	WriteOffHistoryRule	Unit	EP: no history / written-off / settled late
FR-05	A rejected application shall state a typed rejection reason, not a boolean	origination	Decision, RejectionReason	Unit	EP: one case per rejection reason
FR-06	Risk grade A/B/C shall adjust the rate by a fixed spread and cap the approved amount	origination	RiskGrade, Decision	Unit	EP: one case per grade
FR-07	The system shall generate schedules for flat-rate, reducing-balance and bullet products	products	FlatRateProduct, ReducingBalanceProduct, BulletProduct	Unit	Golden file: 10 hand-computed schedules per product
FR-08	Instalment dates shall follow the disbursement day-of-month, clamped for short months	products	DueDateCalculator	Unit	BVA: disbursement on 28th, 29th, 30th, 31st; February in leap and non-leap years
FR-09	The instalments of a schedule shall sum exactly to principal plus total interest	products	ScheduleAssembler	Unit (property)	Property-based: for all valid amount/tenure/rate combinations
FR-10	A grace period shall defer the first instalment while still accruing interest	products	GracePeriodPolicy	Unit	EP + BVA: grace of 0 / 1 / 3 months
FR-11	Repayments shall be applied in the order penalty → fees → interest → principal, oldest instalment first	ledger	RepaymentAllocator, Interest	Unit	Basis path (CC ≈ 11) + EP on payment size

ID	Requirement	Module	Design element (planned)	Test approach	Test design technique
FR-12	A partial payment shall be applied as far as it reaches, leaving the remainder outstanding	ledger	RepaymentAllocator	Unit	BVA: 0 / 0.01 / due – 0.01 / exactly due
FR-13	An overpayment shall become an advance credit unless it clears the loan, in which case early settlement applies	ledger	RepaymentAllocator, SettlementPolicy	Unit	BVA: due + 0.01 / exact payoff / payoff + 0.01
FR-14	A posted ledger entry shall be immutable; corrections occur only as compensating entries	ledger	LedgerEntry, CompensatingEntryService	Unit + inspection	Inspection checklist item + unit test asserting no mutator exists
FR-15	Every posting shall write balanced debit and credit lines	ledger	CoreRepaymentPosting	Unit (invariant)	Invariant test: Σ debits = Σ credits after every posting
FR-16	Penalty shall accrue at 0.05% per day on overdue principal, capped at 10% of the overdue amount	delinquency	PenaltyAccruer	Unit	BVA: at the cap boundary; EP on days overdue
FR-17	Accounts shall be bucketed 0 / 1–30 / 31–60 / 61–90 / 90+ days past due and classified accordingly	delinquency	DelinquencyClassifier	Unit	Basis path (CC $\approx$ 8) + BVA: 0/1/30/31/60/61/90/91
FR-18	A loan more than 180 days past due shall be written off automatically	delinquency	WriteOffPolicy	Unit	BVA: 179 / 180 / 181 days
FR-19	A provision percentage shall be applied per classification	delinquency	ProvisioningPolicy	Unit	EP: one case per classification
FR-20	The system shall produce a customer statement for a date range	reporting	StatementRenderer	Unit	EP: empty range / single entry / full history
FR-21	The system shall report portfolio at risk over 30 days, provision coverage and collection efficiency	reporting	PortfolioReporter	Unit	Golden file: a fixed portfolio with hand-computed metrics
FR-22	Repayments shall be importable from bank statement files in each supported bank's format	ledger	CsvStatementAdapter	Unit + integration	EP: valid row / malformed date / minor-unit amount / duplicate reference

2.3 Quality requirements traced

ID	Verification method	Primary artifact (planned)	Evidence produced at
QR-01	Test	EndOfDayPerformanceTest	M2, M3
QR-02	Test	GoldenScheduleTest, JaCoCo branch report	M2, M3
QR-03	Test + demonstration	PostingFaultInjectionTest	M2, demonstrated M3
QR-04	Analysis	Static-analysis metric report in CI	M2, M3
QR-05	Inspection	Fagan inspection checklist item + LedgerImmutabilityTest	M2
QR-06	Test	EndOfDayConcurrencyTest, 50× nightly	M2, M3

2.4 Coverage of the matrix at M1

Measure	Status at M1
Functional requirements defined	22
Quality requirements defined	6
Constraints recorded	4
Requirements with a planned verification approach	28 of 28 (100%)
Requirements with executed tests	0 — M1 is a planning milestone
Code modules traceable to a requirement	n/a at M1; audited from M2

Audit rule for M2 and M3: every requirement must have at least one test (no test gaps), and every production class must trace to at least one requirement (no rogue code). Both directions are checked, because neither direction can find the other's defect.

3. SQA Plan — Scope and Outline (IEEE 730)

The outline below follows the IEEE 730-2014 structure. Every mandatory section is listed with a one-line plan specific to MicroLend. The full plan is developed across M2 and completed at M3; M1 establishes the scope and the structure.

§	IEEE 730 section	Plan for MicroLend
1	Purpose and scope	This SQAP governs the MicroLend loan-servicing system across M1–M3, covering the five modules and the shared repository, tailored to a three-person Very Small Entity
2	Definitions and acronyms	Define the terms actually used — DPD, PAR30, allocation waterfall, EOD, compensating entry, cyclomatic complexity, EP/BVA — plus error, defect and failure, so the document is self-contained
3	Reference documents	The joint-project guidelines, ISO/IEC 25010, ISO/IEC/IEEE 29148, IEEE 730-2014, IEEE 1028, ISO/IEC 29110, the Business Rules Register and the RTM
4.1	Organisation and independence	Name the three members, their module ownership and the rotating SQA lead. State honestly that organisational independence is impossible in a three-person team, and that it is substituted by non-owner review (no one approves their own pull request) and the external inspector at M2

§	IEEE 730 section	Plan for MicroLend
4.2	Software product risk	Classify MicroLend's product risk as financial, not safety-related ; identify the money path (RepaymentAllocator, DelinquencyClassifier, the ledger) as the high-risk area; set SQA depth from that, since SQA effort is proportionate to product risk
4.3	Tools	Java and JUnit 5, SQLite, Gradle, GitHub and GitHub Actions, JaCoCo for coverage, Checkstyle and a static analyser — each named with the quality question it answers
4.4	Standards, practices and conventions	Constraints C-01 to C-04; the commit convention <code>test(red): / feat(green): / refactor::</code> ; the documentation standard; and the 100% decision coverage on the money path policy
4.5	Effort, resources and schedule	Approximately 39 student-days of construction plus about 8 student-days of SQA work per milestone, mapped onto the Week 6 / 10 / 13 dates with named owners
5.1.1	Evaluate plans for conformance	At each milestone, check that the M1 proposal, the test plan and this SQAP still describe what is actually being built; record deviations
5.1.2	Evaluate product for conformance	The Fagan inspection of the two money-path classes, static-analysis triage, and black-box and white-box test execution against the RTM requirement IDs
5.1.3	Evaluate product for acceptability	Define the M3 acceptance gate: all functional requirements traced to passing tests, $\geq 60\%$ coverage of core logic with 100% decision coverage on the money path, zero open critical defects
5.1.4	Evaluate product life-cycle support for conformance	Check at each baseline that the README, setup instructions, schema DDL and design documents match the code
5.1.5	Measure products	Defect density per module, branch coverage, cyclomatic complexity per method, open and closed defect counts — the metrics the GQM questions consume
5.2.1	Evaluate life-cycle processes for conformance	Verify that the red-green-refactor sequence, pull-request review and merge gates were genuinely followed, using commit and pull-request history as the evidence
5.2.2	Evaluate environments for conformance	Confirm all three members build and test on the same toolchain versions, and that CI is the single source of truth for a green build
5.2.3	Evaluate subcontractor processes	Declared not applicable, with justification: no subcontractors and no third-party components beyond the JDBC driver and test libraries. The standard requires the declaration, not silence
5.2.4	Measure processes	Inspection rate in LOC per hour, defect-removal effectiveness by phase, and rework as a percentage of total effort

§	IEEE 730 section	Plan for MicroLend
5.2.5	Assess staff skill and knowledge	Record that concurrency, white-box test design, static analysis and the Fagan inspection process are not yet taught at M1, and name the self-study and pairing that closes each gap before the technique is used
6.1	Contract review	Treat the joint-project guidelines as the agreement: a checklist mapping every Section 3.1 and 3.2 minimum to where it is met, re-checked at each milestone
6.2	Quality measurement	The GQM programme — one goal, three questions, and the metrics listed in §5.1.5 and §5.2.4
6.3	Waivers and deviations	Any departure from this plan is logged with reason, approver and date — for example, dropping a planned pattern is recorded, not silently abandoned
6.4	Task repetition	A re-inspection is triggered if an inspection finds more than a set number of major defects; the full regression suite re-runs on every push
6.5	Risk to performing SQA	The honest risks to the quality work itself — exam weeks compressing the inspection, the external inspector falling through, one member carrying the SQA load
6.6	Communications strategy	A weekly 15-minute session where each member explains their module's invariants to the other two; defect log and corrective actions in the repository; milestone status summarised in the M2 and M3 documents
6.7	Non-conformance process	How a defect or process non-conformance is raised, triaged by severity, assigned, corrected and verified — the corrective-action loop closed at M2 and M3
7.1	SQA records: collect, file, maintain	All quality records live in /qa in the repository: inspection log, defect log with corrective-action status, static-analysis reports, coverage reports, GQM data and risk-register versions
7.2	Availability of records	Records are versioned with the code in the hosted repository, so any record can be produced on request at the demo

3.1 Inspection planning note (for M2)

The Fagan inspection process runs in six stages: **plan** → **kickoff** → **individual document checking** → **logging meeting** → **edit** → **follow-up and exit**. The planning guideline is **100–200 lines of code per hour** for source code.

That rate is the number to plan with. Inspecting `RepaymentAllocator` plus `DelinquencyClassifier` — roughly 250 lines — means a logging meeting of about 1.5 to 2 hours, which is schedulable with an external inspector and which also provides the denominator for the defect-detection-rate metric in the GQM programme. A team that ignores the rate “inspects” 800 lines in forty minutes and finds nothing, and the defect log then shows exactly that.

4. Cost of Quality and Quality-Culture Statement

4.1 The model

Total cost of quality is the sum of the **cost of conformance** (what is spent to prevent and detect defects) and the **cost of non-conformance** (what defects cost once they exist).

4.2 Expected costs for this project, in student-hours

The currency here is time, because that is the resource the team actually spends. Figures are estimates made at M1 and compared against actuals at M3 — the comparison is itself a GQM metric.

Category	Activity for MicroLend	Estimated effort (student-hours)
Prevention	Writing the Business Rules Register and freezing scope	6
	Coding standard and Checkstyle configuration (constraints C-01 to C-04)	4
	Hand-computing 30 golden schedules in a spreadsheet before coding	8
	Design review of the class model before implementation	6
	Setting up CI, branch protection and quality gates	5
	Prevention subtotal	29
Appraisal	Writing and maintaining the test suite (test-first)	60
	Pull-request review by a non-owner	18
	Fagan inspection of the money path, including preparation	12
	Static-analysis runs and triage	8
	Appraisal subtotal	98
Internal failure	Rework of defects found by tests, review or inspection before submission	25 (expected)
External failure	Rework of defects found by the grader at a milestone demo, or by the following milestone	10 (target: as close to zero as possible)
	Non-conformance subtotal	35
	Total expected cost of quality	162 student-hours

4.3 What the team expects these numbers to show

Three things follow from this table, and each is a prediction the team is prepared to be held to at M3:

1. **Appraisal dominates, and that is correct for this project.** The money path has no safety net other than tests: a wrong instalment does not crash, it quietly produces a wrong number. Detection effort is where the risk is.
2. **Internal failure cost rising is a good sign, not a bad one.** If the register shows more hours of rework triggered by our own tests and inspections at M2 than at M1, it means defects are being caught inside the team rather than at a demo. The number to minimise is **external** failure, not internal. A team reporting zero internal failure cost is reporting that it is not looking.
3. **The 1:10:100 principle is why the 8 hours on golden schedules are spent first.** A rounding rule corrected while writing the spreadsheet costs minutes. The same defect found after schedules, ledger entries, delinquency buckets and portfolio reports have all been built on it requires reworking every one of them, under deadline. Prevention is bought at the cheapest point on the curve.

4.4 Quality-culture statement

The team commits to four behaviours, each with a mechanism that makes it checkable rather than aspirational:

Commitment	The mechanism that enforces it
Quality is not a phase at the end. Quality activities run in the same milestone cycle as the construction work they assess	SQA deliverables are owned per milestone and rotate; the M2 inspection targets M2 code, not a frozen artifact
No one marks their own homework. Every change is reviewed by someone who does not own that module	Branch protection requires one approving review from a non-author; CODEOWNERS routes the request
Defects are data, not blame. A defect found in inspection is a success for the process	The defect log records type, severity and phase found — never who wrote it. The metric is defect-removal effectiveness, not individual defect counts
If it is not evidenced, it did not happen. Every claim in this plan maps to an artifact in the repository	All quality records live in /qa, versioned with the code and reproducible from a milestone tag

The honest limit: in a three-person team, the person reviewing is also a developer on the project. Independence is structural, not organisational, and that is stated in SQAP §4.1 rather than hidden.

5. Standards Awareness

Software has no laws of nature to stop a defect from executing. A silent rounding error or a lost update in our ledger meets no physical resistance the way an under-designed arch meets gravity, so process standards serve as the surrogate laws — and they are a **prevention cost** in the cost-of-quality model rather than paperwork. By business model, MicroLend is *custom software written in-house*: the category where standards are adopted voluntarily to stop chronic rework rather than imposed by a client.

ISO/IEC/IEEE 12207 tells us *what processes must exist*. Of its supporting processes we genuinely perform Configuration Management (branch protection, the m1/m2/m3 baseline tags), Verification and Validation, Joint Review and Quality Assurance. **IEEE 730-2014** tells us *how* the SQA function executes inside that lifecycle: its activities in three groups — SQA process implementation, product assurance, process assurance — its principle that the depth of SQA is set by **product risk**, and its rule that any activity not performed must be declared *not applicable with a justification* rather than quietly dropped. That rule is why our SQAP §5.2.3 explicitly marks subcontractor evaluation as not applicable instead of omitting it.

Full 12207 and CMMI adoption would be wrong at our scale. Three students on one project is a **Very Small Entity** under ISO/IEC 29110's 25-person definition, and at roughly two person-months of effort we fall formally in the **Entry** profile, though we follow the **Basic** profile's structure as taught — its two processes, Project Management and Software Implementation, map one-to-one onto our three milestones, and its Deployment Packages supply ready-made templates and traceability tables so our time goes into the work rather than into inventing document formats. Lightweight here means *scaled to context*, not undisciplined.

Against **CMMI-DEV** we claim **Maturity Level 2 behaviours only**, and we name them: Requirements Management through the RTM, Configuration Management through protected branches and baselines, Process and Product Quality Assurance through the Fagan inspection and static-analysis triage, and Measurement and Analysis through the GQM programme. We explicitly do **not** claim Level 3, because a maturity level describes an organisation's standard processes rather than a single project's, and the statistical process control of Levels 4 and 5 is meaningless with three people and one data point per milestone.

From **ISO 9001:2015** we take two things concretely. The **PDCA cycle** is literally our milestone loop: plan the milestone, build it, measure coverage and defect density, then change the *process* rather than promising to be more careful. **Risk-based thinking** is why our register already records that only one member fully understands the allocation waterfall, although nothing has failed yet. **ISO/IEC 90003** is what licenses us to translate ISO 9001's abstract "design and development verification" into the specific software acts we actually perform — code inspection, static analysis, automated unit tests and decision-coverage criteria.

Finally, rigour is scaled to consequence. MicroLend is not safety-critical, so the domain standards for airborne systems, railway applications and medical devices do not apply, and we make no such claim. But because it is a financial ledger we borrow their one transferable idea — that criticality dictates technique — and adopt as a stated SQAP policy **100% decision coverage on the money path** (RepaymentAllocator, DelinquencyClassifier), together with the coding-standard rule banning double for monetary values: our scaled-down equivalent of a safety standard's language restrictions, recorded under IEEE 730's "standards, practices and conventions" clause rather than left as an aspiration.

What we are not claiming, stated plainly: CMMI Level 3 or above; conformance to any safety-critical standard; organisational independence of the QA function; and certification of any kind. Every standard named above is used as a source of practice, not as a badge.

6. Initial Risk Register

The full register is maintained as a living document at `qa/risk-register.md` in the repository, updated at every milestone and whenever a risk materialises or closes. It is reproduced here as at M1.

6.1 Scoring

Risk exposure is **computed, not asserted**: $E = P \times C$, where P is probability (1 = rare ... 5 = almost certain) and C is consequence (1 = negligible ... 5 = severe).

Exposure	Priority	Response
17–25	Critical	Act now; mitigation owned and tracked weekly
10–16	High	Mitigation planned and started this milestone
5–9	Medium	Monitor; mitigate if exposure rises
1–4	Low	Accept; review each milestone

6.2 The register at M1

ID	Risk (condition → consequence)	Type	P	C	E	Priority	Mitigation	Owner	Status
R1	Money or date arithmetic in schedule generation or the allocation waterfall is subtly wrong → every downstream figure is wrong and the defect is found late	Product	4	5	20	Critical	Hand-compute 30 golden schedules before coding and commit them as the first tests; BigDecimal only, enforced by constraint C-01; property test that Σ instalments = principal + total interest	Hassan Khalid	Open
R2	The team starts M1 late → the compressed schedule pushes work into the M2 window, and M2 carries 35%	Project	5	3	15	High	Freeze scope at five modules with no new features; draft M1 documents from the existing analysis; set up repository and tooling in parallel with document writing	Tughrul Hussain	Open

ID	Risk (condition → consequence)	Type	P	C	E	Priority	Mitigation	Owner	Status
R3	The external inspector required at M2 is not arranged in time → the inspection criterion (20% of SE431 M2) cannot be fully evidenced	Process	3	4	12	High	Request the instructor's team pairing immediately after M1; agree a date in Week 8; prepare the inspection package in advance	Muhammad Ibrahim	Open
R4	Commits are squashed or tests are written after the code → the red-green-refactor trail cannot be reconstructed, losing TDD marks at M2 and M3	Process	3	4	12	High	Squash-merge disabled in repository settings from day one; commit convention enforced; merges use --no-ff; the pull-request template asks for the red commit hash	Tughrul Hussain	Open
R5	Concurrency defects in the end-of-day engine appear only under load → late, hard-to-reproduce failures near M3	Product	3	4	12	High	Immutable value objects; per-account striped locks with ascending lock ordering; a unique (loan_id, business_date) idempotency key; deterministic latch-driven tests with no sleep	Muhammad Ibrahim	Open

ID	Risk (condition → consequence)	Type	P	C	E	Priority	Mitigation	Owner	Status
R6	Individual contribution is not attributable → individual evidence suffers, which is 20% of each member's milestone mark	Project	3	3	9	Medium	Module ownership equals directory ownership; every change by pull request reviewed by a non-owner; SE431 deliverable ownership rotates each milestone; AI Usage Log maintained continuously	Tughrul Hussain	Open
R7	Several M2-graded techniques are not taught until after they must be designed for → the team designs ahead of the lectures and gets it wrong	Project	4	4	16	High	Design the structures now, defer detailed technique work until taught; start the four non-reconstructable evidence streams at M1; re-check each design against the lecture when delivered	Muhammad Ibrahim	Open

ID	Risk (condition → conse- quence)	Type	P	C	E	Priority	Mitigation	Owner	Status
R8	AI-assisted work is submitted that a member cannot explain in the viva → reduced or zero individual marks, and a possible integrity concern	Project	3	5	15	High	AI Usage Log updated continuously; the module owner reviews and rewrites generated code in their own module; dry-run Q&A inside the team before each demo	Hassan Khalid	Open

6.3 Review schedule

- **M1:** register initiated with eight risks, scored and owned.
- **M2:** every risk re-scored using evidence from the architecture, inspection, static analysis, testing and SCM; risks arising from implementation added; residual exposure recorded.
- **M3:** continuous updating from M1 to M3 demonstrated; mitigation effectiveness evaluated against actual project data; residual risk recorded.

7. Summary of M1 Quality Position

Deliverable	Status at M1
Quality requirements	6 defined, each with a measurable target and a named verification method; 2 characteristics declared not applicable with justification
Constraints	4 recorded, C-01 enforced by the build
RTM v1	22 functional and 6 quality requirements, each with a planned verification approach and a named test design technique
SQAP	Scope set and all mandatory IEEE 730 sections outlined with project-specific plans; one section declared not applicable with justification
Cost of Quality	Estimated at 162 student-hours with the prevention/appraisal/failure split and three stated predictions
Standards awareness	12207, IEEE 730, 29110, CMMI-DEV, ISO 9001 and 90003 related to this project's scale, with explicit non-claims
Risk register	8 risks, computed exposure, owners assigned, living document in the repository

What M1 deliberately does not yet contain, because the techniques are taught later in the semester and the rubric does not require them at this milestone: executed test results, the inspection defect log, the static-analysis report, coverage figures and GQM data. Each has a planned home in this document and an owner, and the four evidence streams that cannot be reconstructed retrospectively — the TDD commit trail, SCM baselines, the static-analysis baseline and GQM data collection — have been started at M1 even though they are assessed later.

Annex A — AI Usage Log

Required by the joint-project guidelines §9. Milestones 2 and 3 carry their own logs.

Tool used throughout: Claude Opus 5 (Claude Code). **All Milestone 1 entries dated 4 October 2026.**

#	Purpose	Material adopted	Modifications made	Verification
1	Compare candidate project ideas against both rubrics	Comparative analysis; MicroLend chosen over two alternatives	Scope cut to a locally-run application with no deployment or paid services	Each rubric minimum traced to where it is met; team accepted the choice
2	Draft the SE423 document	Class model, SOLID analysis, pattern justifications	Diagram split into three views for legibility; CLI replaced by a local web interface	Found and corrected a Simple Factory presented as a Factory Method
3	Draft the SE431 document	Quality requirements, RTM, SQAP outline, CoQ, risk register	Four items reclassified from quality requirements to constraints	Targets and effort estimates checked as realistic for a three-person team
4	Repository setup and coding standard	Checkstyle config, CODEOWNERS, PR template, branch rules	C-01 check rewritten as a token rule after a false positive	Branch protection tested by a direct push, which was blocked and logged
5	Business Rules Register	43 numbered rules with boundary values	Numeric thresholds reviewed and accepted unchanged by the team	Boundary values cross-checked against the RTM test design
6	Build and CI scaffold	Gradle build, Money, MoneyTest, GitHub Actions workflow	Written as a real TDD cycle, red commit before green	CI green: compile, 10 tests, Checkstyle, coverage
7	Submission PDFs	Build script and title page	Fonts changed to carry mathematical symbols	Pages inspected; a fault dropping $\leq$ and $\times$ was found and fixed

Declaration for Milestone 1

I confirm that I have read, understood and verified the material submitted under my name, that I can explain and defend the design decisions, quality requirements and analysis it contains, and that any AI-assisted content was reviewed and modified by me where needed.

Muhammad Ibrahim — 2023446 · origination, delinquency; SQAP outline, risk register

Signature: Date:

Hassan Khalid — 2023242 · products, reporting; quality requirements, RTM

Signature: Date:

Tughral Hussain — 2023532 · ledger; standards awareness, Cost of Quality, repository setup

Signature: Date:

Team rules for AI use

1. Generated code enters the repository only through the module owner, who reviews it and answers for it.
2. No generated test is accepted without reading what it asserts.
3. Business rules and numeric thresholds are the team's decisions, not defaults.
4. Entries are added when the work happens, not before a deadline.
5. Anything a member cannot explain is rewritten or withdrawn before submission.